Delta Lake: Advanced Guide for Rust Developers

Deep dive into the Delta Lake protocol, concurrency control, deletion vectors, CDC, and custom Rust implementations.

---

1. The Delta Lake Transaction Protocol

The Delta Lake protocol is based on optimistic concurrency control (OCC). Each commit contains an atomic set of actions that are either applied completely or rejected on conflict. Conflicts are detected by checking the latest commit version: if someone else wrote a new commit while you were preparing yours, your commit is rejected and you retry with the new base snapshot. This mechanism guarantees serializable isolation without heavy locks.

1.1 JSON Commit Structure

Each commit file in _delta_log/ contains a JSON array of actions. The key action types are defined in the protocol specification and implemented in delta-rs as Rust structs with full serde serialization:

Action	Rust Struct	Purpose
Protocol	Protocol	Min reader/writer versions and enabled features
Metadata	TableMetadata	Schema, partitions, config, createdTime
Add	Add	New file: path, size, stats, partitionValues, deletionVector
Remove	Remove	Deleted file: path, deletionTimestamp, dataChange
CommitInfo	CommitInfo	Commit metadata: operation, user, metrics, time
CDC	CDC	Change Data Feed entry for stream processing
DomainMetadata	DomainMetadata	Third-party system metadata (Iceberg compat)

1.2 Checkpoints and Log Compaction

A checkpoint is a Parquet file containing all Add/Remove actions for the current version. Instead of reading every JSON file from version 0 to N, the library loads the last checkpoint and only reads subsequent JSON commits. The _last_checkpoint file has a JSON with version and size fields for instant lookup.

By default, a checkpoint is created every 10 commits (configurable). Log compaction merges many JSON files into one for faster loading. In delta-rs, these are available through protocol::create_checkpoint() and protocol::cleanup_expired_logs_for(). Checkpointing is atomic and does not block parallel writes to the table.

1.3 Commit Strategies

The library supports two commit strategies. CommitOrBytes::TmpCommit uses atomic rename (copy-if-not-exists) via a temporary file. CommitOrBytes::LogBytes uses conditional put with version check. The choice depends on storage: S3 uses LogBytes with DynamoDB for coordination, local FS uses TmpCommit via atomic rename. Each commit also generates a unique operation_id (UUID) for tracking and CustomExecuteHandler support.

2. Concurrency Control Mechanisms

Delta Lake implements several levels of locking for safe parallel access. The main mechanism is optimistic concurrency control (OCC), but the protocol also defines conflict resolution rules for different operations.

2.1 How OCC Works

With OCC, each reader records the table version it bases its work on. When trying to write, it checks that the base version has not changed. If it has changed, a conflict occurs and the operation retries with the new base version. In delta-rs, this is implemented in the LogStore trait through write_commit_entry(), which returns TransactionError::ConcurrentModification on conflict. You can configure the number of retry attempts.

2.2 Conflict Resolution Rules

The Delta Lake protocol defines detailed rules for resolving conflicts between different operation types. Not all conflicts cause errors:

Operation A	Operation B	Result
Append	Append	Both succeed (safe)
Append	Optimize	Append retries with new version

Optimize	Optimize	Second retries with new version
MERGE	MERGE	Conflict - second retries
UPDATE	DELETE	Conflict - second retries
Write	Schema Change	Conflict - schema changed
OPTIMIZE	Vacuum	Safe (independent operations)

2.3 DynamoDB Commit Coordinator (S3)

When working with Amazon S3, the default approach uses try-retry with conditional put. This can cause livelock under high contention. Delta Lake supports DynamoDB as an external commit coordinator. DynamoDB provides atomic compare-and-swap through ConditionalWrite, guaranteeing progress even under high load. In delta-rs, configure this via storage options with dynamodb_locking_enabled=true.

3. Deletion Vectors

Deletion Vectors (DV) allow you to delete and update rows at the file level without rewriting the entire Parquet file. A DV is a compact bitmap (Roaring Bitmap) with bits set for deleted rows. Physically, DVs are stored in separate files in the table directory and referenced from the Add action via the deletionVector field.

3.1 How Deletion Vectors Work

When you run UPDATE or DELETE, instead of rewriting the Parquet file, Delta Lake creates a new Deletion Vector. When reading, the library checks if a file has a DV, and if so, skips the rows marked in the DV. This is like a soft-delete at the storage level without changing the data file. The old Parquet file stays on disk, but the deleted rows are invisible to readers.

3.2 DeletionVectorDescriptor Structure

In delta-rs, a DV is described by DeletionVectorDescriptor with fields: storageType (INLINE, FULL_FILE, or PARTIAL_FILE), pathOrInlineDv (path or inlined data), offsetInFile, sizeInBytes, and cardinality (number of deleted rows). This feature requires explicit activation via table.add_feature() and needs minimum writer version 4.

3.3 Performance

DV greatly reduces I/O on UPDATE/DELETE: instead of rewriting an entire file (which can be 256 MB+), only a small DV (usually kilobytes) is written. However, reading has overhead from loading and applying DVs. It is recommended to run OPTIMIZE regularly to merge files with DVs into clean files without gaps. The compaction threshold is configurable.

4. Change Data Feed (CDC)

Change Data Feed (CDF) captures all changes in a table (insert, update, delete) as stream records. CDF lets you build pipelines for syncing data between systems, implement event sourcing, and track data lineage.

To enable CDF, set the table property delta.enableChangeDataFeed = true. After this, each commit with dataChange=true generates CDC files (Parquet) with columns _change_type (insert/update/delete_preimage/update_postimage) and _commit_version.

4.1 Reading CDC in delta-rs

The library reads CDC through the table.cdf() method which returns RecordBatch with changes for a given version range. Each record has the change type, old and new values. This is useful for incremental loading into downstream systems, data replication, and building materialized views.

```
// Read CDC between versions 10 and 20
let changes = table.cdf("10", "20").await?;
for batch in changes {
  let change_type = batch.column_by_name("_change_type");
  println!("Changes: {:?}", change_type);
}
```

5. Row Tracking and Column Mapping

Row Tracking gives each row a unique row_id that survives MERGE, OPTIMIZE, and other file rebuilds. The row_id is stored in the Add action through the base_row_id field. This is critical for Deletion Vectors: when files are compacted, DVs are redistributed, and row_id ensures correct mapping.

Column Mapping uses named columns instead of positional ones. This lets you rename and drop columns without rewriting data files. Instead of a column position number in the Parquet file, it uses a unique columnId from the table metadata. This provides reliable schema evolution when multiple processes work on the same table. Feature flag: columnMappingMode = 'name'.

6. Custom LogStore Implementation

The delta-rs architecture lets you implement custom storage backends through two traits: ObjectStoreFactory and LogStoreFactory. ObjectStoreFactory creates an object_store instance for reading/writing data files. LogStoreFactory creates a LogStore for managing the transaction log.

Registration uses a global registry based on DashMap with lazy initialization via LazyLock. The LogStore factory is determined automatically by the URI scheme: s3:// -> S3LogStore, az:// -> AzureLogStore, gs:// -> GCSLogStore. For a custom scheme (e.g., mystore://), register the factory through object_store_factories().insert() and logstore_factories().insert().

```
use deltalake::logstore::logstore_factories;
use std::sync::Arc;

// Register your custom storage backend
logstore_factories().insert(
  "mystore://".parse().unwrap(),
  Arc::new(MyCustomLogStoreFactory)
);

// Now you can open tables with your scheme
let table = open_table("mystore://my-bucket/table").await?;
```

7. Integrations and Ecosystem

Delta Lake integrates with many compute engines and tools. delta-rs serves as the bridge between the Rust ecosystem and the Delta Lake format.

Tool	Integration Type	Key Features
Apache DataFusion	TableProvider trait	SQL queries, DataFrame API, predicate pushdown
Apache Arrow	Native support	Zero-copy columnar, RecordBatch exchange
Polars	LazyFrame scan	Read via delta-rs -> Arrow -> Polars
DuckDB	Parquet reading	Indirect integration through data files
Python (PyO3)	deltalake pip package	Full API, accessible via PyArrow
Iceberg (UniForm)	Metadata compat	Read Delta tables with Iceberg engines

UniForm (Universal Format) is especially interesting: it adds Iceberg-compatible metadata to Delta checkpoints, allowing Delta tables to be read by Iceberg engines without conversion. This makes tables accessible to both formats at the same time, which is very useful in multi-engine environments.

8. Performance Tuning

8.1 Data Skipping

Delta Lake stores min/max statistics for each column in the Add action (stats field). During a table scan, the library skips files whose statistics do not match the query predicate. For maximum efficiency, use Z-ORDER on frequently filtered columns and collect statistics with the configuration setting delta.dataSkippingStatsColumns.

8.2 I/O Caching with Foyer

delta-rs supports I/O caching through the delta-cache feature flag, based on the foyer library. The cache stores frequently used Parquet file fragments and JSON commit segments in memory with LRU eviction. This is very effective for repeated queries on the same partitions and frequent transaction log scans. Cache size is configurable through DeltaTableConfig.

8.3 IO Runtime Configuration

You can configure a custom tokio runtime for I/O operations through DeltaTableConfig.io_runtime. This is useful when integrating into existing applications that already have their own runtime. By default, a separate runtime with multi-threaded scheduler is created. You can also set log_buffer_size for storage operation parallelism and log_batch_size for the batch size when processing the log.

9. Contributing to delta-rs

The delta-rs architecture is modular and well-structured. The main areas for contribution include: new storage backends (LogStore implementations), operation optimizations, protocol parsing improvements, and DataFusion integration enhancements. The project uses a Cargo workspace with many crates under crates/. Tests are consolidated into two binaries (it and it_datafusion) to speed up compilation.

Key code principles: trait-based abstractions for extensibility, Arc<dyn Trait> for shared ownership, builder-pattern for the API, thiserror for type-safe error handling, serde for protocol serialization, and feature-gates for dependency management. When adding a new storage backend, implement ObjectStoreFactory, LogStoreFactory, and register them through the global registries.

1. Study delta_kernel: delta-rs delegates protocol parsing to the official delta_kernel crate. Understanding the kernel helps when working with actions and snapshots.

2. Use tracing: The library uses the tracing crate for structured logging. Enable DEBUG level to understand internal behavior during troubleshooting.

3. Profile your storage: The main bottleneck is I/O. Use metrics to monitor storage operation latency and optimize accordingly.

4. Test with concurrency: Write integration tests with parallel writes to verify OCC works correctly in your environment.

5. Monitor checkpoints: Regularly check checkpoint status. A missed checkpoint degrades table loading performance over time.
